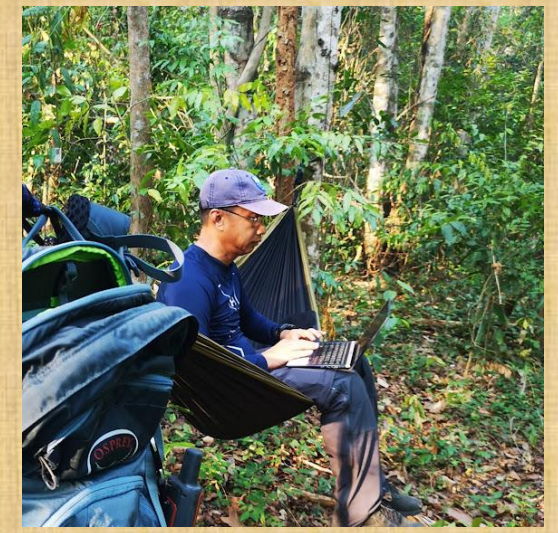
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS



Lecture 5 · Linked Lists





Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
Mahidol University
Faculty of Information and Communication Technology

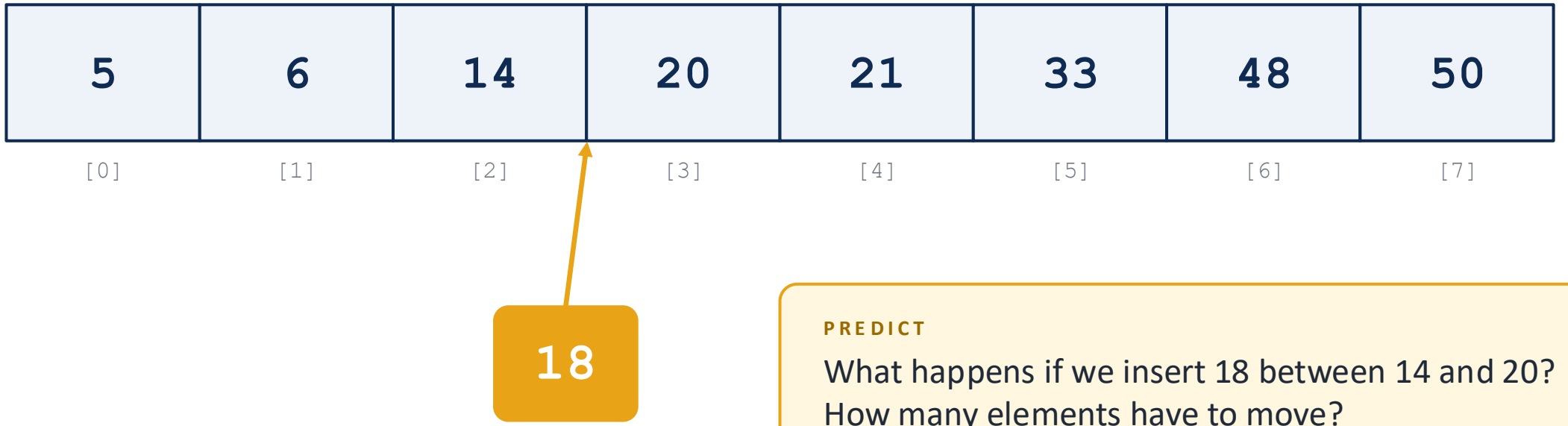
2026/1

Learning Outcomes

1. Explain why linked lists are useful compared with arrays
2. Explain the roles of data, next, prev, first/head and last/tail
3. Trace references during insertion, deletion, search and traversal
4. Implement fundamental linked-list operations in Java
5. Handle empty-list, one-node, first, middle, last and key-not-found cases
6. Analyze the time complexity of linked-list operations
7. Distinguish singly linked, singly + tail, and doubly linked lists
8. Explain how head + tail references give an O(1) FIFO queue
9. Choose an appropriate structure for a simple application

The Problem with Arrays

a sorted int array (capacity 10, size 8)



PREDICT

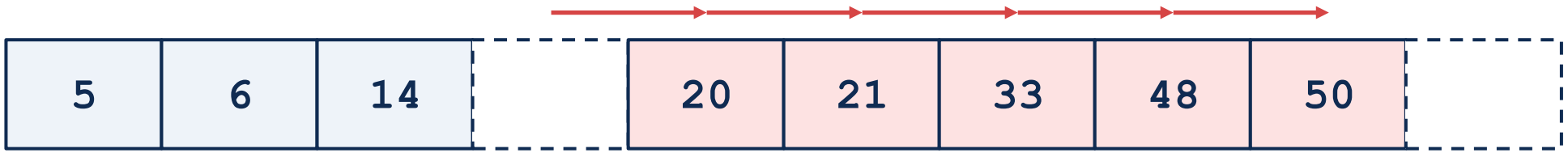
What happens if we insert 18 between 14 and 20?
How many elements have to move?

Insertion in an Array Means Shifting

Step 0 · before



Step 1 · shift 50, 48, 33, 21, 20 one slot to the right (5 moves, from the back)

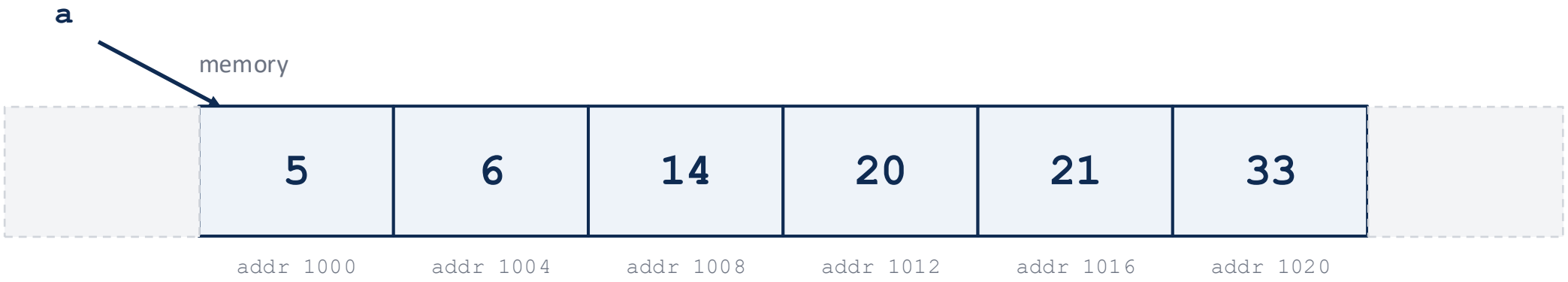


Step 2 · write 18 into index 3



Insertion may require O(n) movement. Inserting at index k shifts n − k elements. Deletion is the same in reverse. And if the array is full, we must allocate a bigger one and copy everything.

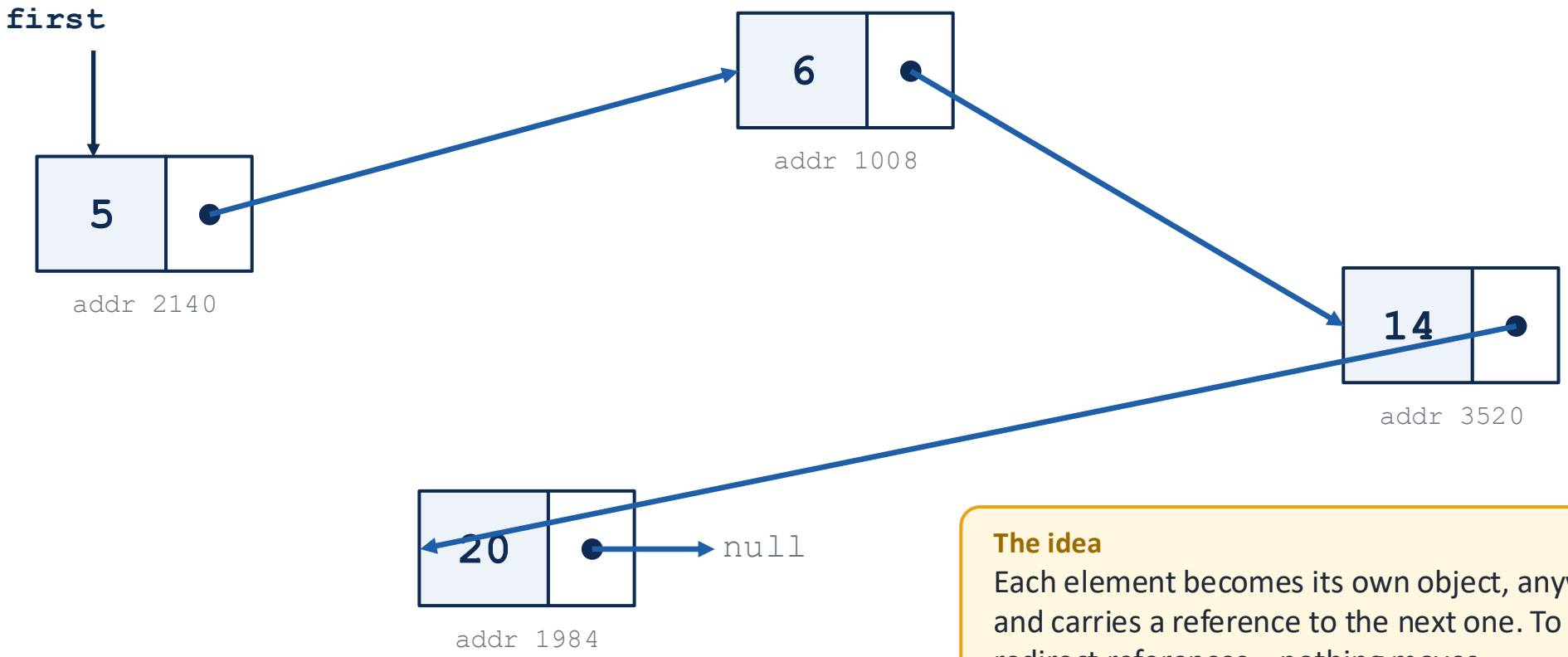
Array Memory Model: Contiguous



- Elements occupy contiguous (adjacent) logical memory locations.
- Address of a[i] = base + i × elementSize, so access by index is O(1).
- The block is allocated once with a fixed capacity.
- To insert or delete in the middle, neighbours must physically move.

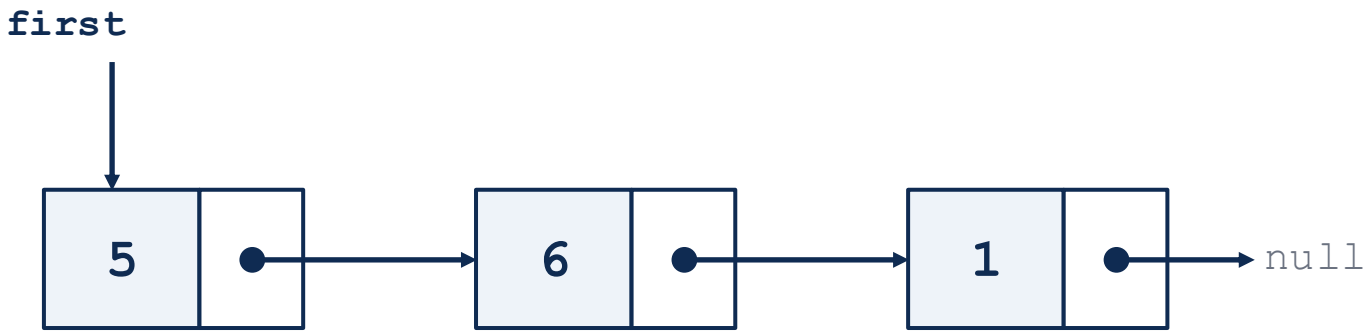
The trade-off
Contiguity gives fast random access, but it is exactly what makes insertion and deletion expensive.

What If Data Did Not Need to Be Adjacent?



The idea
Each element becomes its own object, anywhere in memory, and carries a reference to the next one. To insert, we only redirect references—nothing moves.

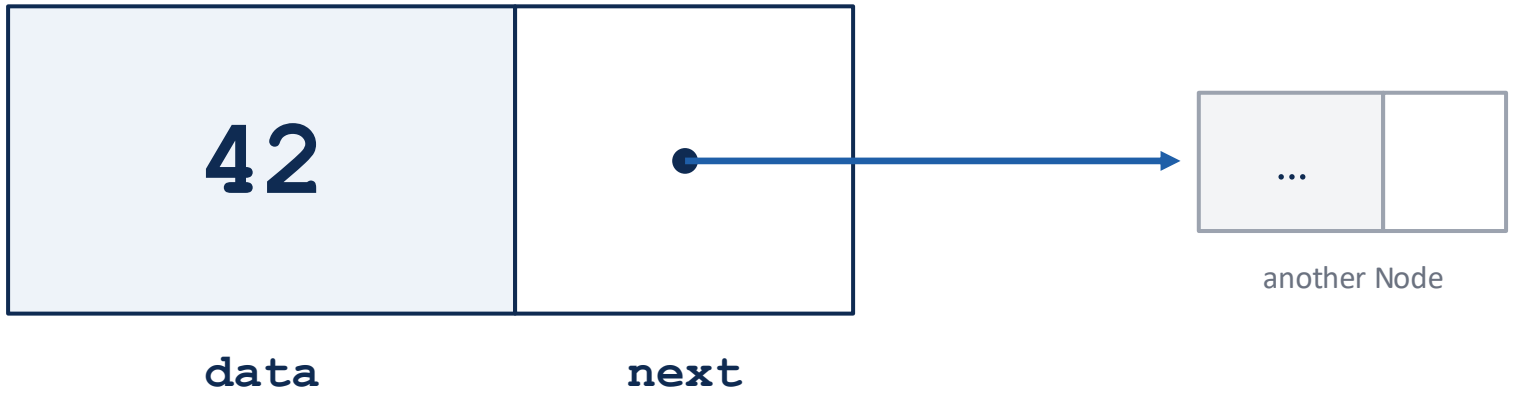
Linked List Mental Model




```
first → [5 | next] → [6 | next] → [1 | next] → null
```

- A linked list is a chain of nodes.
- Each node stores a value (data) and a reference to the next node (next).
- A single reference, first, gives access to the whole list.
- The last node's next is null: the end of the chain.

Anatomy of a Node



`data`
The value stored in this node. Here an int; in general any type.

`next`
A reference to another Node object—or null if there is none. It does not contain the node; it points to it.

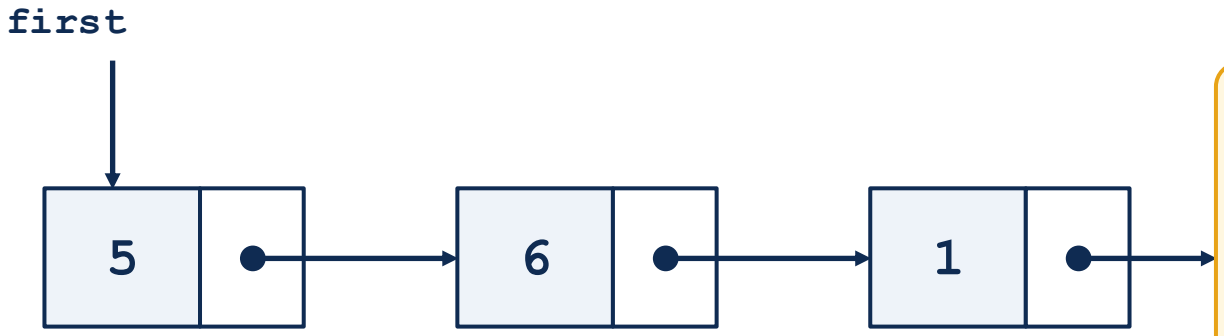
Java: the Node Class

```
class Node {
    int data;
    Node next;

    Node(int data, Node next) {
        this.data = data;
        this.next = next;
    }
}
```

- Two fields: data (the value) and next (a reference to a Node).
- The constructor sets both fields in one step—handy for building lists.
- new Node(7, null) creates a node holding 7 that points nowhere.
- A class can have a field of its own type: this is what makes a chain possible.

Build This List



TRACE

Using only new Node(data, next), write Java that builds 5 → 6 → 1 → null.
Hint: which node must exist before the others can point to it?

One solution (build from the back):

```
Node third  = new Node(1, null);
Node second = new Node(6, third);
Node first  = new Node(5, second);
```

Or in one expression (inside-out):

```
Node first =
    new Node(5,
      new Node(6,
        new Node(1, null)));
```

What Does first Mean?

```
class LinkedList {
    Node first;    // the first node

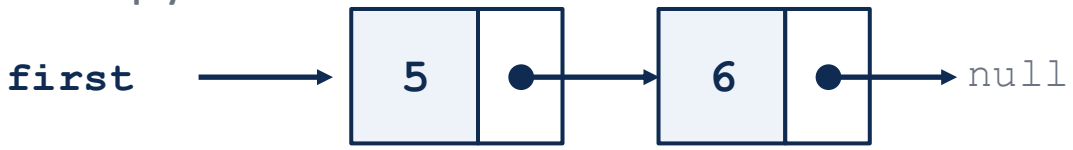
    LinkedList() {
        first = null;   // empty list
    }

    boolean isEmpty() {
        return first == null;
    }
}
```

Empty list

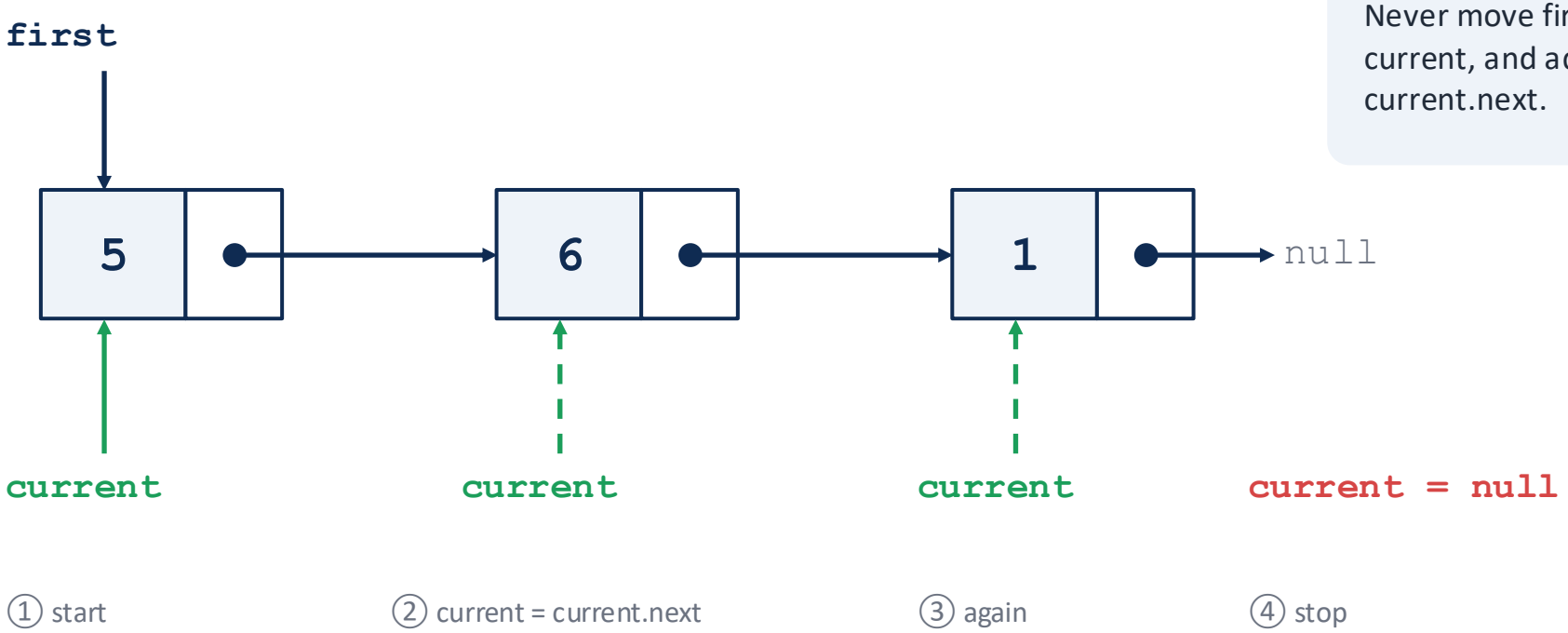


Non-empty list



Invariant
first == null ⇔ the list is empty.
Every operation must keep this true. Some textbooks call this reference head.

Traversing a Linked List



① start

② current = current.next

③ again

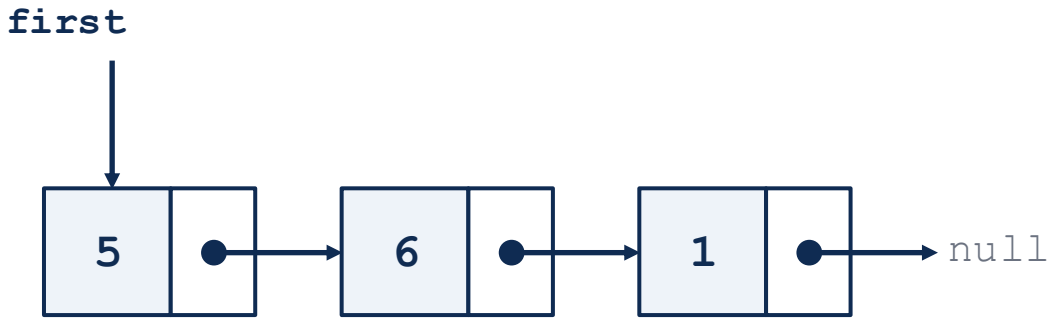
④ stop

Rule of traversal
Never move first. Use a separate reference, current, and advance it with current = current.next.

Trace This Loop

```
Node current = first;

while (current != null) {
    System.out.println(current.data);
    current = current.next;
}
```



TRACE

What values will be printed, and in what order?
How many times is the condition current != null evaluated?

Trace table

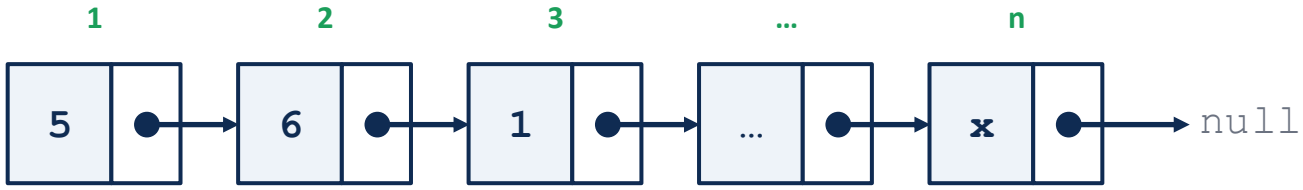
5
6
1

4 times

Traversal Complexity

O(n)

to visit every node

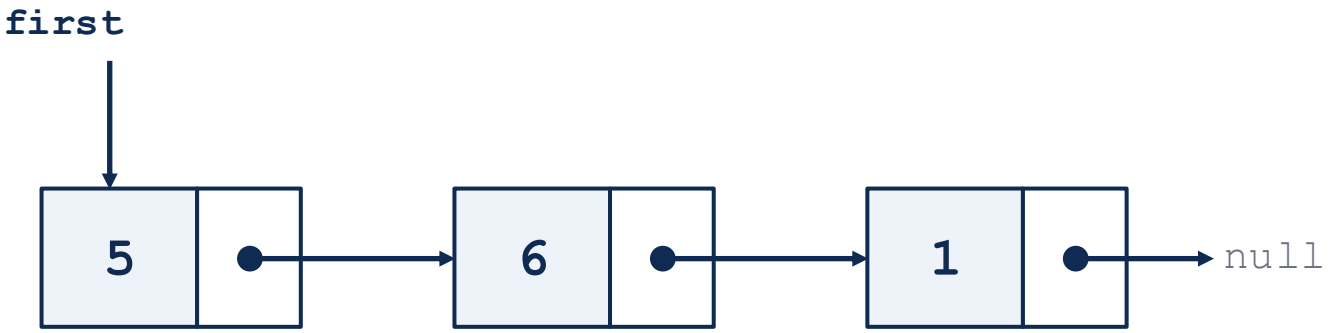


each step of the loop does a constant amount of work

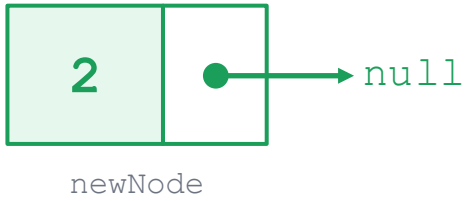
- There is no address formula: node i can only be reached by following i references from first.
- Access by position (get(i)) is therefore O(n), unlike an array's O(1).
- Search for a key is also O(n): we may have to inspect every node.
- Rule of thumb: count how many nodes an operation must visit.

Insert 2 at the Front

Before



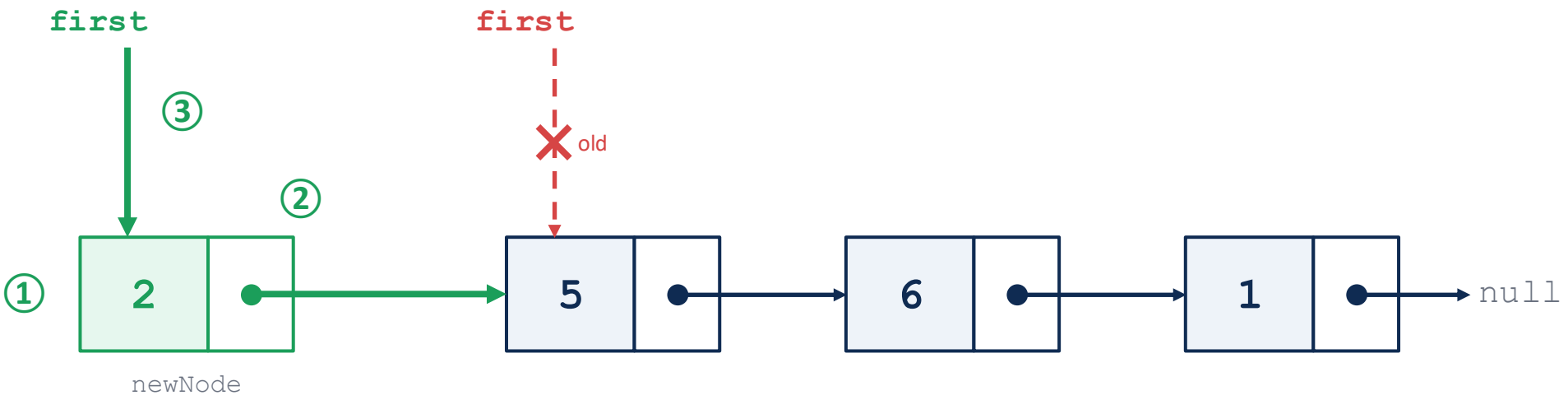
New node (not yet linked)



PREDICT

Which references must change so that 2 becomes the first node? How many assignments do you need? Does the order matter?

Reference Update, One Step at a Time



1. newNode = new Node(2, ...) create the node
2. newNode.next = first ← link to old first node
3. first = newNode ← move first (old arrow disappears)

After

`first → 2 → 5 → 6 → 1 → null`

No existing node moved. Node 5 is unchanged—only what points to it changed.

Java: addFirst()

Step by step

```
public void addFirst(int data) {
    Node newNode = new Node(data, null);
    newNode.next = first;
    first = newNode;
}
```

Idiomatic (the constructor does step 2)

```
public void addFirst(int data) {
    first = new Node(data, first);
}
```

- On the right, the argument first is evaluated before the assignment, so the new node captures the old first node, then first is updated.
- Return type is void: the method changes the list and returns nothing.
- Works on an empty list: new Node(data, null) becomes the only node.

Why Is addFirst O(1)?

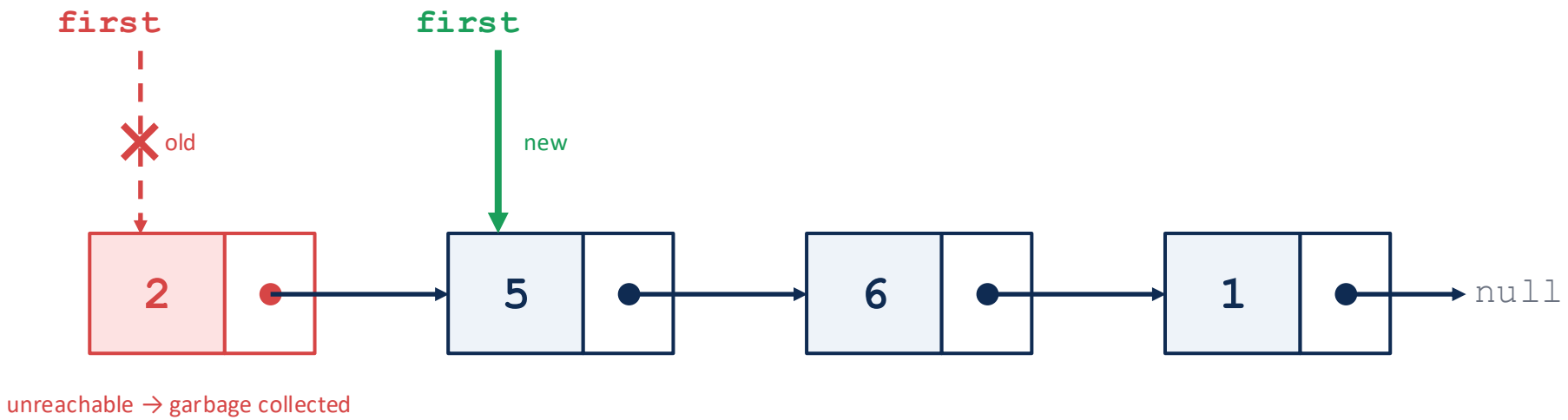
COMPLEXITY

Does the algorithm depend on the number of nodes in the list?

null

never touched · could be n − 1 nodes

Delete the First Node




```
first = first.next;
```

One assignment, O(1)
first now refers to node 5. Node 2 still exists for a moment, but nothing references it any more, so Java's garbage collector will reclaim it. We never call free()—but we also never 'delete' a node directly; we make it unreachable.

Java: removeFirst()

FIND THE BUG

What happens when we run `first = first.next;` on an empty list?

first is null, so first.next throws NullPointerException. Guard it:

```
public void removeFirst() {
    if (first != null) {
        first = first.next;
    }
}
```

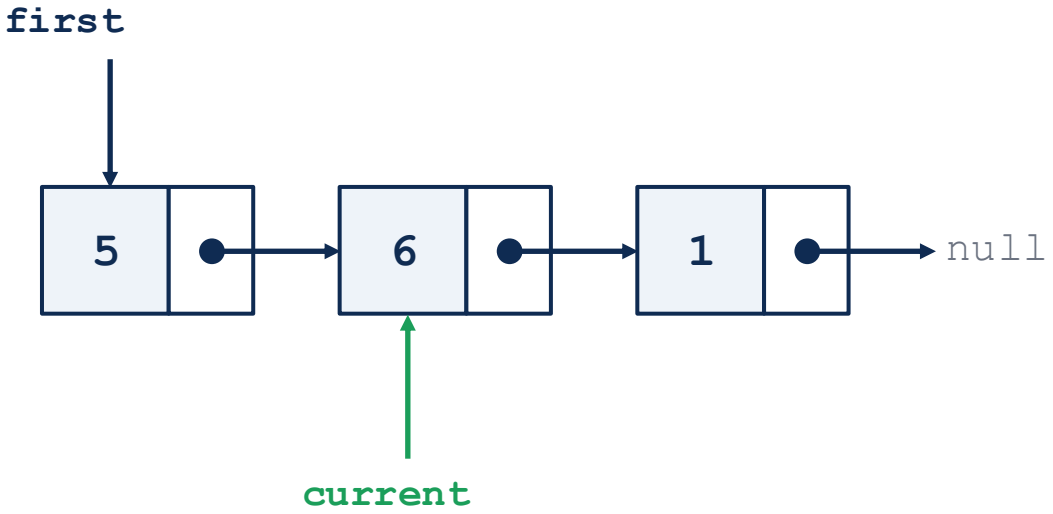
Variant that returns the removed value

```
public int removeFirst() {
    if (first == null) {
        throw new NoSuchElementException();
    }
    int value = first.data;
    first = first.next;
    return value;
}
```

Both are O(1). The one-node case needs no special code: first.next is null, so the list simply becomes empty.

Searching for a Key

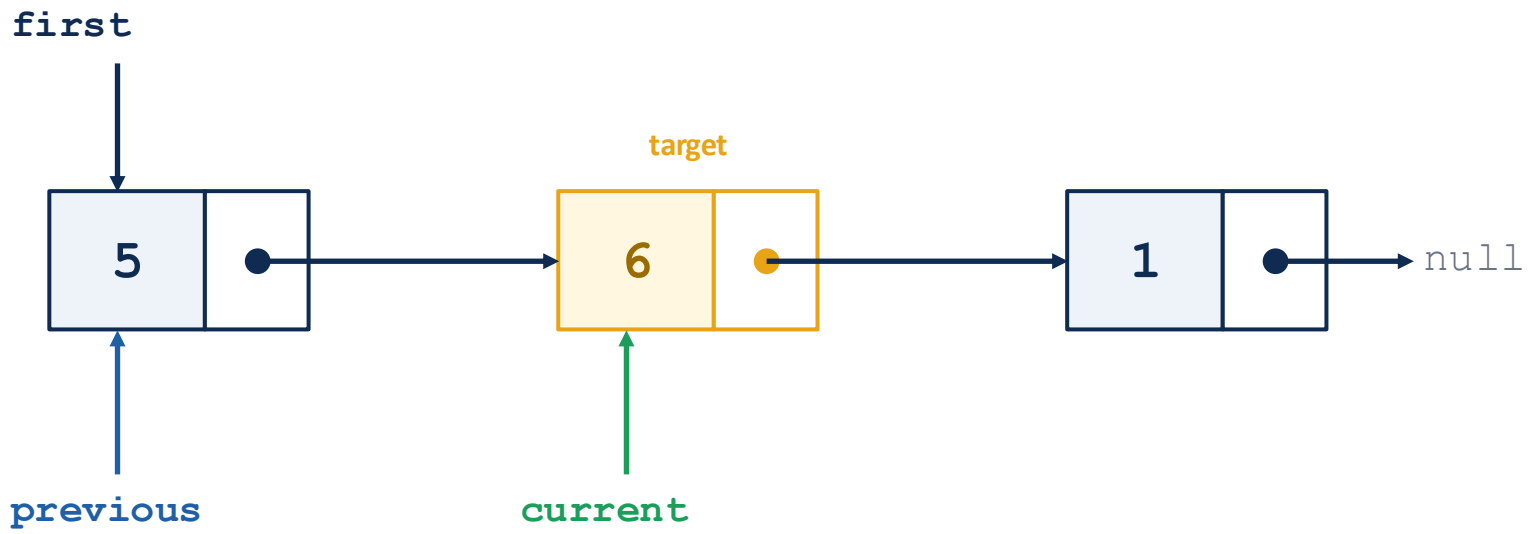
```
public boolean contains(int key) {
    Node current = first;
    while (current != null) {
        if (current.data == key) {
            return true;    // found
        }
        current = current.next;
    }
    return false;           // not found
}
```



key = 6 → stop here, return true

Same shape as traversal
Early exit on a hit; falling off the end (current == null) means 'not found'. Worst case visits all n nodes: O(n).

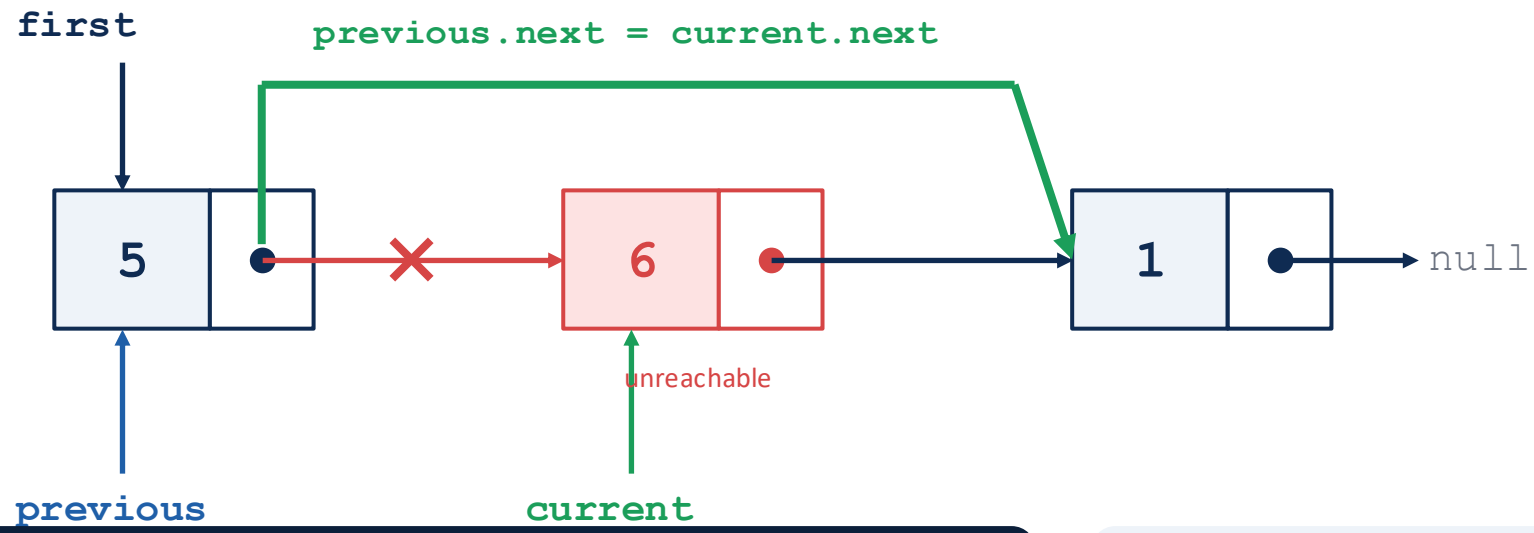
Delete Key = 6: Track Two References




```
Node previous = null;
Node current  = first;
while (current != null && current.data != key) {
    previous = current;
    current  = current.next;
}
```

Why two references?
A node cannot see who points to it. To bypass current we must edit previous.next, so previous trails one step behind.

Bypass the Node




```
previous.next = current.next;    // unlink 6
```

Result: first → 5 → 1 → null
Node 6 still points to 1, but nothing points to 6. The garbage collector reclaims it. Cost after the search: O(1).

Delete-Key Edge Cases

Empty list
first == null → nothing to do (or throw). current is null from the start.

Target is the first node
previous == null → cannot do previous.next. Use first = first.next instead.

Target is in the middle
previous.next = current.next (the general case).

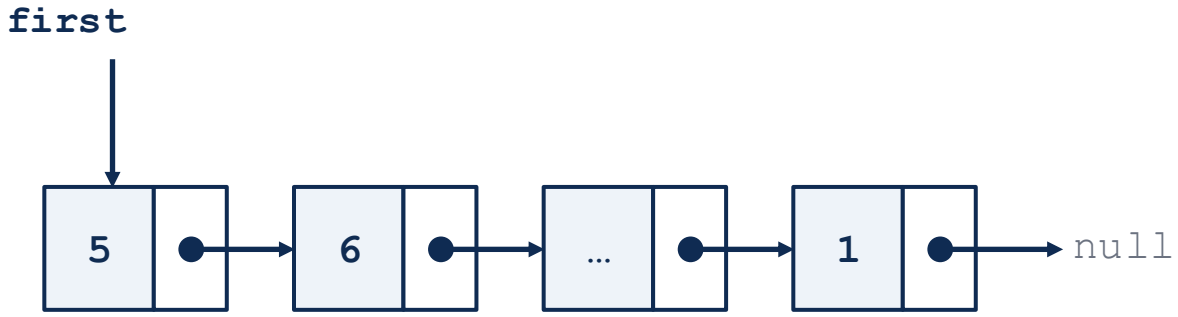
Target is the last node
Same as middle: current.next is null, so previous.next becomes null. Works unchanged.

Key does not exist
Loop ends with current == null → do nothing / return false. Never dereference current.

```
public boolean removeKey(int key) {
    Node previous = null, current = first;
    while (current != null && current.data != key) {
        previous = current;
        current  = current.next;
    }
    if (current == null) return false;   // absent
    if (previous == null) {
        first = first.next;              // first
    } else {
        previous.next = current.next;    // mid/last
    }
    return true;
}
```

Singly Linked List: What Is Fast?

Operation	Complexity
access by position	O(n)
search	O(n)
add first	O(1)
delete first	O(1)
add last	O(n)
delete last	O(n)



...we must first walk all the way from first

COMPLEXITY

Why is adding at the end still O(n)? We are not shifting anything—so where does the time go?

Can We Reach the Last Node Faster?

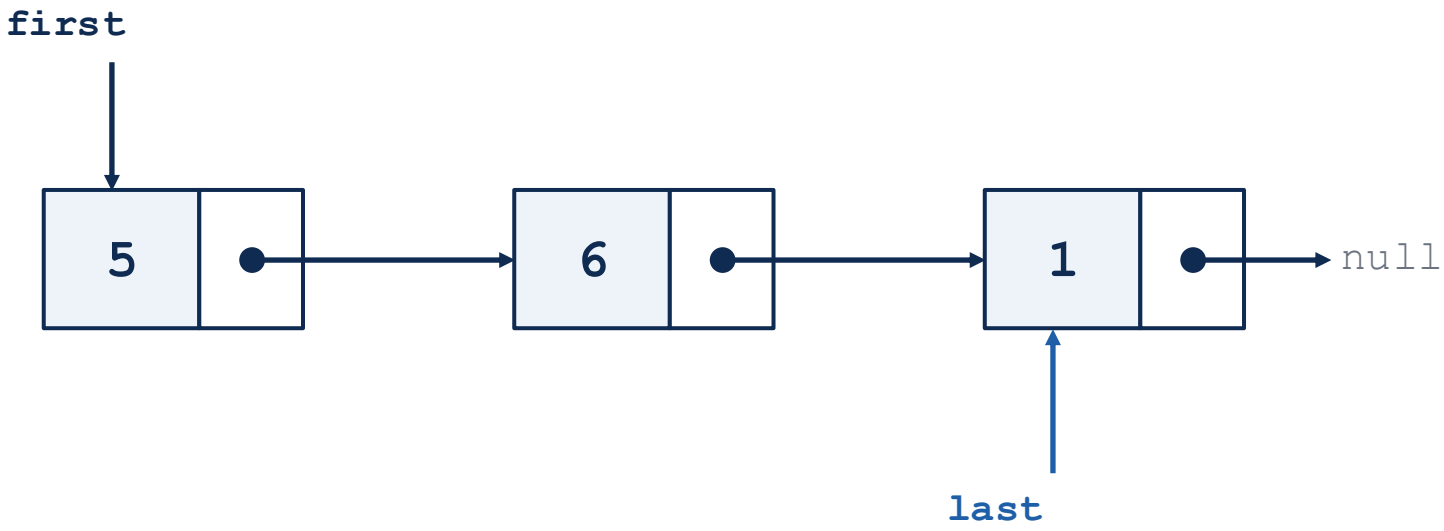
Idea: keep a second reference that always points to the last node.

```
class LinkedList {
    Node first;   // head (or null)
    Node last;    // tail (or null)
}
```

Terminology
Some textbooks call this a double-ended linked list. It is NOT a deque (double-ended queue), which is an abstract data type, not a node structure.

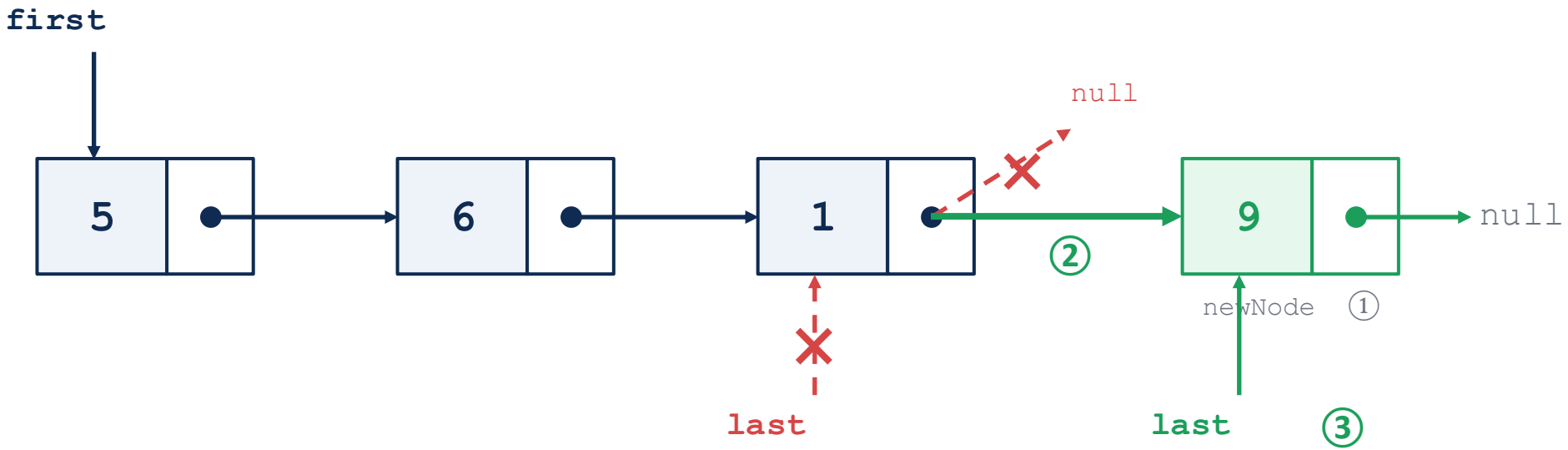
- Cost: one extra reference in the list object (not per node).
- Benefit: last gives direct access to the final node—no walking.
- Responsibility: every operation that could change the last node must now update last as well. This is the new invariant we must maintain.

Singly Linked List with Head + Tail References



last gives direct access to the final node. The nodes are still singly linked: each has only next. Invariants: last.next == null, and first == null ⇔ last == null.

Insert 9 at the End



1. `newNode = new Node(9, null)`
2. `last.next = newNode` (old null link is replaced)
3. `last = newNode` (move the tail reference)

O(1) — no walking, three assignments

But What If the List Is Empty?

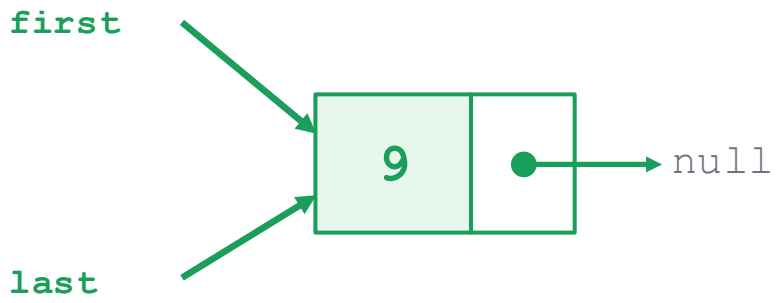
FIND THE BUG

last.next = newNode; last = newNode;
Run this on an empty list. What happens?

last is null → last.next throws NullPointerException, and first is never set.

```
if (first == null) {     // empty
    first = newNode;
    last  = newNode;
} else {                 // non-empty
    last.next = newNode;
    last = newNode;
}
```

Empty list: both references must be set



Pattern to remember
Whenever a structure has two 'entry' references, the transitions 0 ↔ 1 nodes are special: both references must change together. A one-node list has first == last.

Java: addLast()

```java
public void addLast(int data) {
    Node newNode = new Node(data, null);
    if (first == null) {        // empty
        first = newNode;
    } else {
        last.next = newNode;    // after old tail
    }
    last = newNode;             // new tail
}
```

```java
public void removeFirst() {
    if (first == null) return;
    first = first.next;
    if (first == null) last = null;  // now empty
}
```

Checklist

✓ empty list handled
✓ one-node list: first == last afterwards
✓ last always updated
✓ last.next == null still true
✓ O(1): no loop

Note the small change to removeFirst: when the list becomes empty, last must be reset too—or it dangles on a removed node.

A Queue Is FIFO: First In, First Out

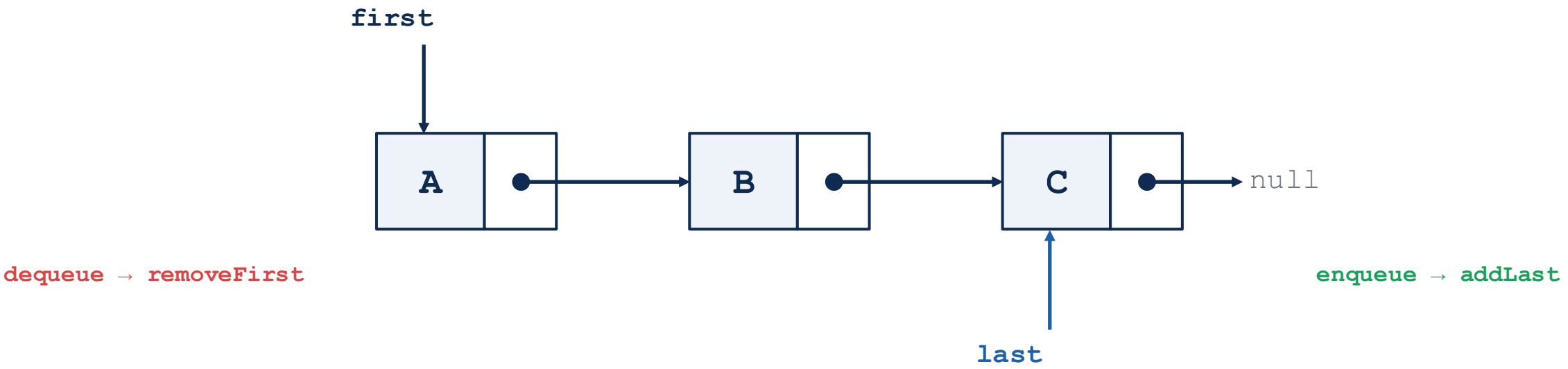
A arrived first, so A leaves first.



Not LIFO!
Last In, First Out describes a stack (push/pop at one end). A queue is the opposite discipline: the oldest element leaves first.

Where queues appear
Print jobs, task schedulers, breadth-first search, request buffers, customers waiting in line.

Queue Using a Linked List with Head + Tail



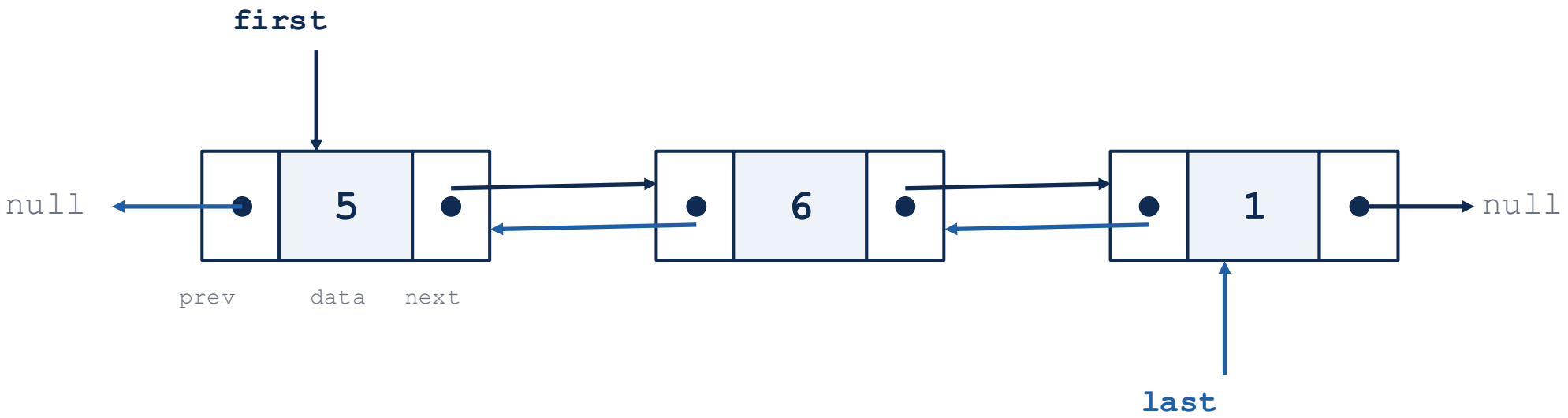
Queue operation	List operation	Complexity
enqueue(x)	`addLast(x)`	O(1)
dequeue()	`removeFirst()`	O(1)
peek()	`first.data`	O(1)
isEmpty()	`first == null`	O(1)

PREDICT

Suppose we used a plain singly linked list (no last). Which queue operation becomes O(n)? Could we fix it by enqueuing at the front instead?

Can We removeLast() in O(1)?

Doubly Linked List



- Each node has three fields: prev, data, next.
- Every link is reciprocal: if x.next == y then y.prev == x.
- We can now walk in both directions, and reach a node's predecessor in O(1).
- Price: one extra reference per node, and twice as many references to keep consistent.

Java: Doubly Linked Node and List

```
class DNode {
    int   data;
    DNode next;   // forward link
    DNode prev;   // backward link

    DNode(int data) {
        // next and prev start as null
        this.data = data;
    }
}
```

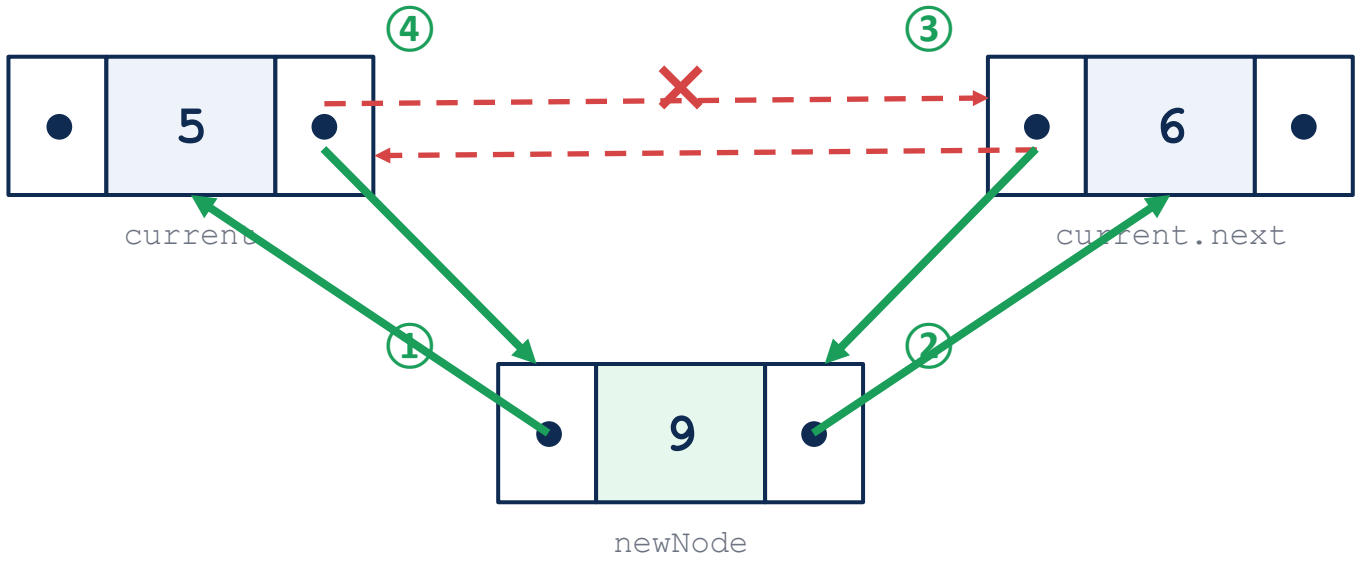
previous node

```
class DoublyLinkedList {
    DNode first;   // head, or null
    DNode last;    // tail, or null
}
```



The list object keeps both first and last, exactly as before. The difference is inside the nodes.

Insert newNode After current, One Reference at a Time

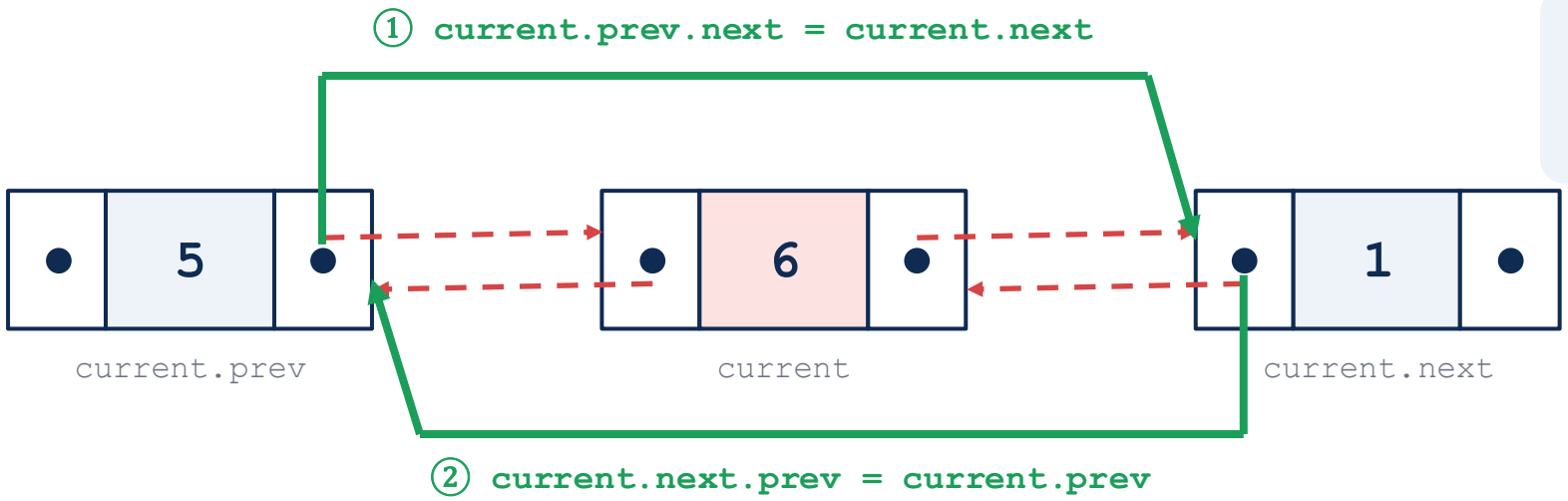



```
DNode newNode = new DNode(9);
newNode.prev = current;        // ①
newNode.next = current.next;   // ②
if (current.next != null)
  current.next.prev = newNode;// ③
else
  last = newNode;     // was tail
current.next = newNode;        // ④
```

Order matters. Set the new node's two links first (① ②) while current.next is still intact. Then repoint the neighbours (③ ④). Doing ④ first would lose the old current.next. Afterwards every link is reciprocal again.

Check: after the four steps, is x.next == y ⇔ y.prev == x true for every pair? Verify on the diagram.

Delete current



No search needed: we reach the predecessor via prev. Deletion of a known node is O(1) — including removeLast.

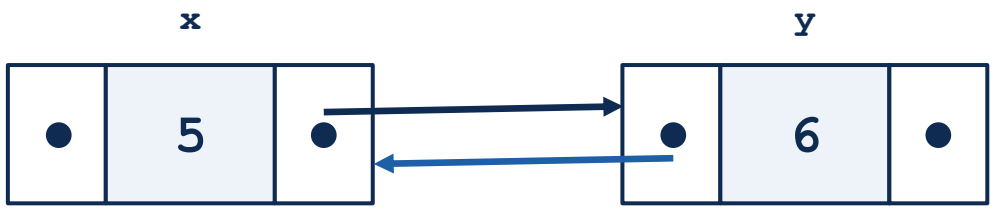
```
current.prev.next = current.next;   // ①
current.next.prev = current.prev;   // ②
```

Find the bug: this code assumes current has BOTH neighbours. If current is first, current.prev is null (① crashes) → set first = current.next instead. If current is last, current.next is null (② crashes) → set last = current.prev instead. A one-node list hits both.

Doubly Linked List Invariants

```
first == null   <=>   last == null          // empty list
first != null   =>    first.prev == null
last  != null   =>    last.next  == null

for every pair of nodes x, y:
    x.next == y   <=>   y.prev == x    // reciprocal
```



`x.next == y  and  y.prev == x`

Why invariants help
Every operation must be true → operation → true.

Debugging tip
Write a checkInvariants() method that walks the list and asserts each rule. Call it after every operation in your tests. A broken prev link is invisible when you only traverse forwards—this catches it.

Design tip
When you write an operation, list which invariants it could break; each one is a case to test.

Complexity Comparison

Operation	Array	Singly linked	Singly + tail	Doubly linked (+tail)
access by index	O(1)	O(n)	O(n)	O(n)
search (unsorted)	O(n)	O(n)	O(n)	O(n)
insert first	O(n)	O(1)	O(1)	O(1)
delete first	O(n)	O(1)	O(1)	O(1)
insert last	O(1)*	O(n)	O(1)	O(1)
delete last	O(1)	O(n)	O(n)	O(1)

* amortized O(1) for a dynamic array with spare capacity; O(n) when the array must grow. Array 'delete last' assumes we only decrement size.

Green = constant, red = linear. Notice how each added reference (tail, then prev) turns one more red cell green — at the cost of more references to maintain.

Which Structure Would You Choose?

A **Frequent random access by index; size rarely changes.**

array

B **Frequent insertion and removal at the front only.**

Singly linked

C **O(1) insertion and deletion at both ends (a deque).**

Doubly linked